

Efficient use of compilers

Michelle Shaw
CRC, ACEnet

Outline

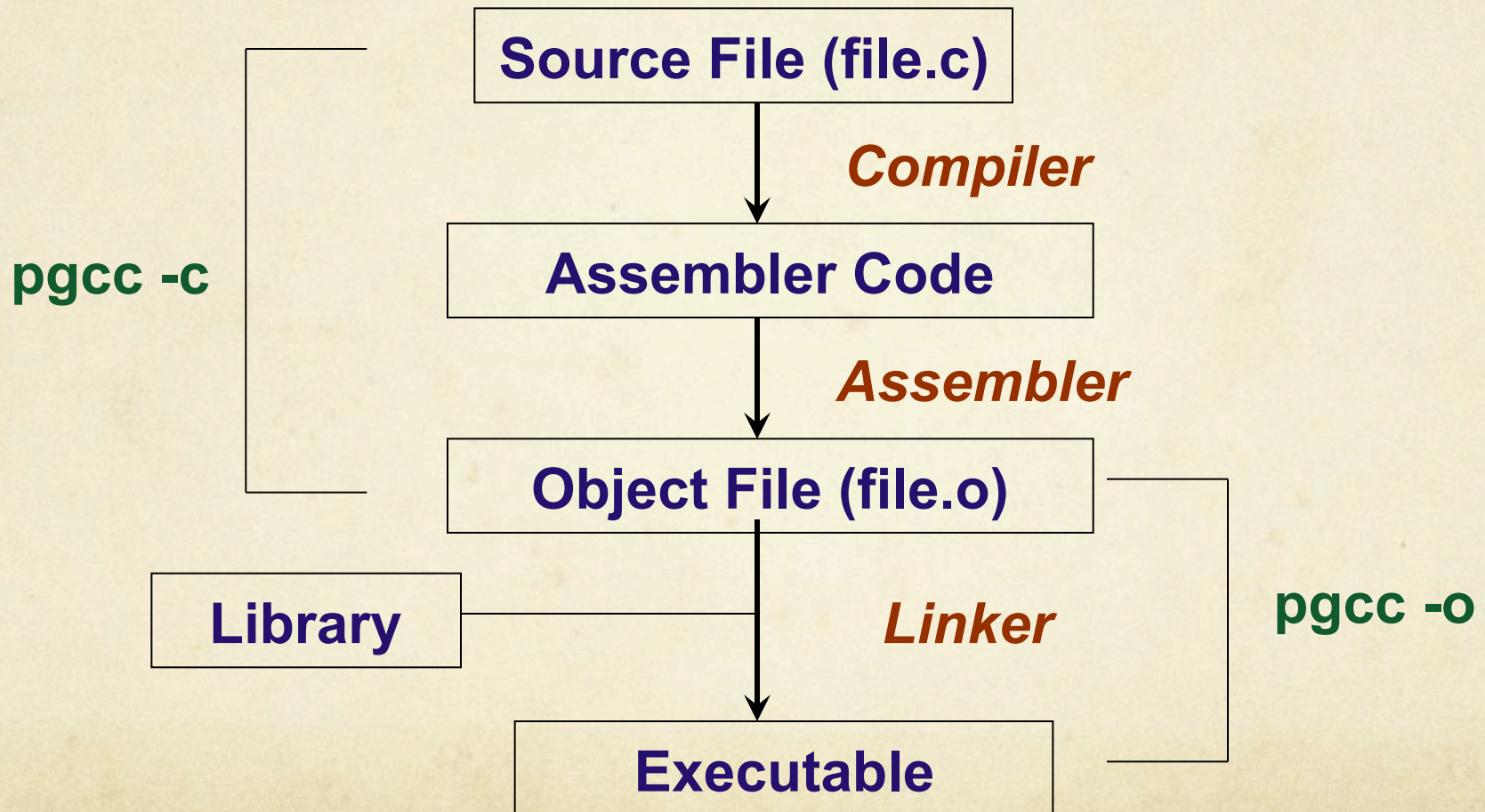
- What I am assuming you know
- Compilers we have at ACEnet
- What a compiler does
- Different stages of a compile process
- Common flags used in many compilers.
- Useful knowledge
- Optimization
- Compiling parallel code

Assumed Knowledge

- You've turned something code-ish into an executable, your own code or otherwise, which means you should then know:
 - What the term “compiler” refers to
 - What the terms “source” and “binary” refer to
- You know your way around a linux box...

Compiler Basics

Building the Executable From Source



Compiling

- Translates source into assembler, then into symbols the linker can use to build the executable
- Checks your syntax for errors
- Does not check if the functions you use exists
- A general format for invocation:

compiler_name [options/flags] *filename.ext*

Compilers at ACEnet

- PGI 7.1-5
 - Portland Group compilers
 - Our default and recommended
 - pgcc, pgCC, pgf77, pgf90, ...
- Sun Studio 12
 - cc, CC, f77, f90, ...
- GNU 3 and 4 family
 - GNU open source compilers
 - gcc, g++, gfortran (the old g77 is part of gfortran now)

Flags

- These are options passed to the compiler that change how it compiles and links your code.
- They all start with a *-flagname* and can be followed by a keyword if necessary
- Flags are evaluated left to right, so anything to the right will override those on the left

Flags

- Flags can differ from one compiler to the next
 - Example: `-V` in sun studio is “version” but for GNU 4 family we use `-v` for the same option
- They can be used to specify different levels of optimization, from none to everything
- Some more useful flags:
 - `-c`: compile the source and stop when the object file is created
 - `-o`: combine the object files and libraries and build the specified executable file
 - `-help`, `--help`: print out the most commonly used flags

Linking

- “links” together object files and any additional libraries to give the executable
- This is where it checks for missing function calls
- By default the compiler does the compiling and linking stages one after another

Libraries

- Libraries include code fragments that are needed in the executable and can be inserted at link time (static) or at runtime (dynamic)
- Use of static libraries makes code more portable but makes bulky executables
- Use of dynamic libraries is less portable but yields smaller executables
- A note: check the “bit”-ness of the library and match that with your architecture (i.e. 32 or 64-bits)

Common locations of Libraries

- System-specific libraries are usually in:
 - `/lib` and `/lib64`
- It is unlikely you will need to link these in directly
- Some system libraries related to compilers, graphics, etc. are located in:
 - `/usr/lib` and `/usr/lib64`

Common locations of Libraries

- Program and application libraries are located in:
 - /usr/local/lib and /usr/local/lib64
 - /usr/local/<package>/lib
 - /usr/local/<package>/lib64
- Some additional less common libraries for X11 graphics are in:
 - /usr/X11R6/lib and /usr/X11R6/lib64

Putting It All Together

- Compilation can be done in one step:

pgcc file.c -o file.x

- or in two:

pgcc -c file.c; pgcc -o file.x file.o

- NOTE: the link step can be done by the linux linker “ld”, but I would recommend that you use the compilers instead:
 - They use ld at a low level anyways
 - They know where their own libraries are whereas you may need to explicitly tell ld where to find those

Putting It All Together

- Sometimes you need to explicitly tell the compiler where to find include files...

```
pgcc -I/home/dshaw/includes file.c -o file.x
```

- Sometimes you need to tell the linker where to find libraries...

```
pgcc file.c -o file.x -L/usr/local/FFTW3/lib -lfftw3
```

- Sometimes we need to add the library explicitly...

```
pgf90 file.f90 -o file.x /usr/local/FFTW2/lib/libfftw.a
```

Putting It All Together

- A step back – library linking format?
linker <exe> <objfiles> -L/dir/to/libs -lidentifier
- Where “identifier” refers to the part of the library name lying between “lib” and the .a or .so extension (i.e. “-lfftw3” becomes “libfftw3.so” during linking)
- NOTE: the default is to link libraries dynamically if they exist, and statically otherwise
- To force static linking, either specify the libraries explicitly or add the -static flag

***Errors and How To Find
Them***

Errors

- Compile time

- Link time

- Run time

Compile Time Errors

- These are the easiest to fix.
- Generally mistakes in your code, syntax errors, undefined variables.
- Might get warnings too which, while can be ignored, shouldn't be.
- When fixing errors, *start with the very first one* – often it is the cause of a good amount (or all) of the remaining ones

Linking Errors

- The most common linking error is 'undefined reference to <function name>'
- There are a few causes for this kind of error:
 - Calls to a function without defining it
 - Missing object file or library
 - Symbol mismatch, normally resulting from trying to link in libraries with different symbol table formats

Linking Errors

- Another common linking error comes from building code that is 32-bit on a 64-bit system (or *vice versa*)
- Two possibilities:
 - Incompatible object code
 - Relocatable error (usually discovered when linking in some 32-bit libraries with 64-bit object files...)

Run Time Errors

- Code compiles and links but fails when run
- These are generally a problem on the behalf of the programmer
- Code can crash or give garbage results
- Runtime errors are usually due to a problem in the logic of the code, not the syntax

Run Time, cont'd...

- Crashed programs usually produce a core
 - compile in debug mode with `-g` to make use of them for debugging
- Core file contains “working memory” when crash occurred
- Core files can be analyzed using a debugger like TotalView or gdb
- WARNING: this only tells you where the program crashed, not necessarily where the error occurred

Run Time, cont'd...

- Normally programs die where a problem prevented further execution, not necessarily where the error occurred
- However, this can provide a good starting point when debugging
- With a debugger, the problem can eventually be tracked down

***A Few Brief Words About
Optimization***

Optimization Flags

- There are a number of optimization flags common to many compilers.
- `-ON`
 - Nth optimization level
- `-fast` (pgi, other compilers besides GNU)
 - “macro” that sets several optimization flags
- `-funroll_loops` (gnu):
 - Loop unrolling (don't worry about what it means)
- Many others... check the documentation!

Optimization Levels

- There are a number of optimization levels with each compiler, generally N can be 0 to 5.
- 0 is no optimization, and is not necessarily the default with the compiler.
- 1-5 is increasingly heavier 'optimizations'. Generally 4 and 5 will not help and has a good chance to break your code.
- There are a number of optimizations that you can do by hand that are done by the compilers.

Composite flags

- Composite flags are also called “macros”
- They are shortcuts for a list of other flags
- Most common is “-fast”
- To see what flags are set by a macro, check the man pages for the compile
- Some compilers have a handy option to see what flags are set for these macros:
 - PGI: `pgcc -help -fast`

Many words of warning

- Note: optimization levels invoke several different optimizations
- These differ depending on the optimization level chosen
- Keep in mind that some of these optimizations change your code
- These changes can produce incorrect code!
- Use optimizations with care and be sure you have some idea about what they do

More Information

- Good references:
 - Internet
 - Compiler documentation
 - man pages
- Experimentation teaches you a lot, too